

BananaCare: Banana Farm Monitoring and Farmer Assistance Information System

Problem

Banana farmers may have difficulty keeping organized records of their farm details, production information, disease or pest incidents, and requests for agricultural assistance. Agriculture personnel may also rely on separate or manual records, making it difficult to monitor reported farm conditions, review the history of each farmer, and follow up on pending assistance.

Objectives

To develop a Banana Farm Monitoring and Farmer Assistance Information System that efficiently manages farmer and farm information, reported farm conditions, and agricultural assistance records.

Specific Objectives

1. To provide a system for registering and managing farmer and farm records.
2. To allow farmers to report banana diseases, pest infestations, and other farm conditions.
3. To allow agriculture staff to monitor reported conditions and update their status.
4. To provide an assistance-tracking feature for farmer requests, recommendations, and follow-up actions.
5. To make banana farm information and assistance records easier to access and manage.

System Features

1. Farmer and Farm Profile

Stores the basic information of every registered farmer and farm.

Information may include:

- Farmer's full name
- Contact number
- Farm location
- Farm size in hectares
- Banana variety
- Planting date
- Estimated production

2. Disease and Farm-Condition Monitoring

This feature allows farmers to report observed farm problems and allows the Admin to monitor each report.

Reported problems may include:

- Fusarium Wilt
- Sigatoka
- Pest Infestation
- Weather Damage
- Soil Condition
- Other farm conditions

Report status may be:

- Reported
- Under Monitoring
- Resolved

3. Assistance and Recommendation Tracking

This feature allows farmers to request agricultural assistance and allows the Admin to record progress, recommendations, and completion details.

Assistance may include:

- Seedlings
- Fertilizer
- Training
- Disease-Control Support
- Farm Visit
- Other assistance

Assistance status may be:

- Pending
- In Progress
- Completed

System Requirements

Requirement	BananaCare Implementation
Working User Interface	Python/Flask, HTML, CSS, and Jinja-based BananaCare interface
At least 3 major features	1. Farmer and Farm Profile 2. Disease and Farm-Condition Monitoring 3. Assistance and Recommendation Tracking
Data Input and Processing	Register farmers, update farm profiles, submit condition reports, request assistance, update report status, and record recommendations
Output/Result	Farmer and Admin dashboards, farm records, report histories, assistance status, staff remarks, and recommendations
Input Validation	Required fields, password length, unique username, valid dates, positive farm size, and affected-area limits
Error Handling	Invalid login, duplicate username, missing information, invalid numeric values, unauthorized access, and invalid status updates
At least 3 test cases	Valid input, invalid input/error handling, and edge cases

Technology Used

Technology	Purpose
Python	Used as the main programming language for the application's backend logic.
Flask	Used for routing, request processing, authentication workflows, validation, and rendering system pages.
HTML5	Used to structure the system's pages, forms, tables, navigation, and dashboards.
CSS3	Used to create the BananaCare-themed responsive interface, including layouts, cards, forms, tables, and buttons.
Jinja	Used to display dynamic records and reusable page content inside HTML templates.
SQLAlchemy	Used to define database models and perform database operations through Python.
SQLite	Used as the default local classroom and demonstration database.
PostgreSQL/Pysscopg	Supported as an alternative local database through the configured DATABASE_URL.
Flask-Login	Used to manage user sessions and protect authenticated pages.
Werkzeug	Used to securely hash and verify account passwords.
Python unittest	Used to run automated tests using an isolated in-memory database.

AI Used

ChatGPT was used during the early planning stage to brainstorm an appropriate mini-system topic and generate the initial Markdown proposal. OpenAI Codex was then used throughout system development as an AI-assisted programming, debugging, refactoring, testing, and documentation tool. The development team reviewed the generated suggestions, requested changes where necessary, tested the resulting system, and remained responsible for verifying that the final implementation met the project requirements.

AI PROMPT AND OUTPUT EVIDENCE

Brainstorming and Markdown Proposal Prompt

AI Tool Used: ChatGPT

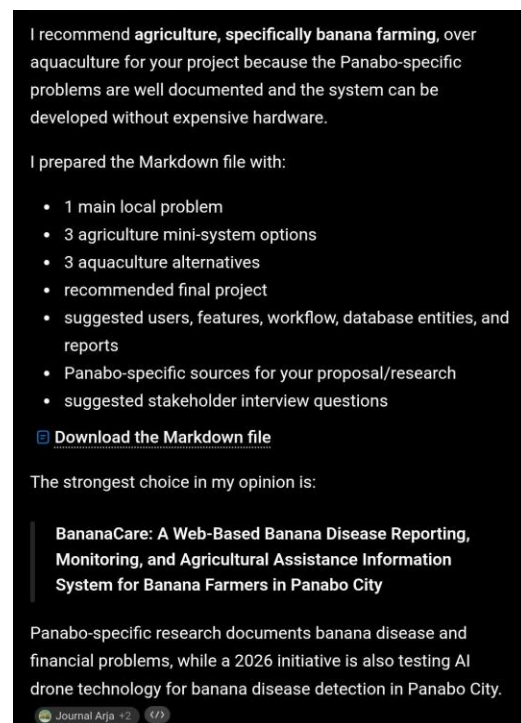
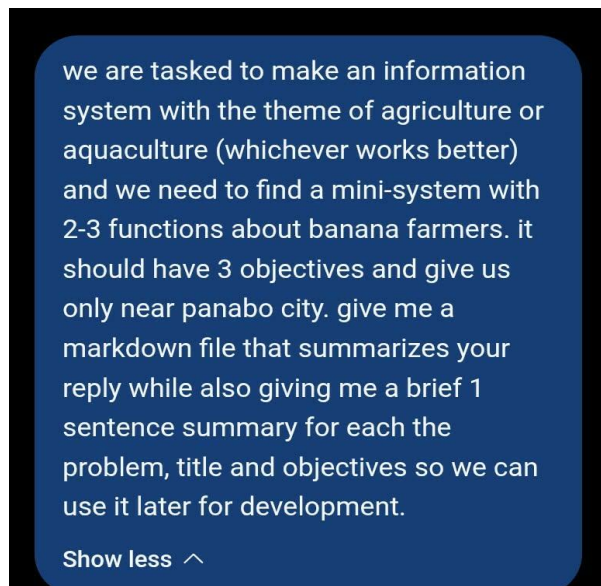
Purpose of the Prompt:

The purpose of this prompt was to brainstorm an appropriate agriculture-related mini information system and organize the selected idea into a clear Markdown proposal. The prompt asked ChatGPT to identify a practical problem, suggest a system title, define three manageable features, and prepare general and specific objectives suitable for the module assessment.

What We Used/Changed from the AI Output:

We selected the suggested Banana Farm Monitoring and Farmer Assistance concept because it provided a clear real-world problem and three features that could be completed within the required project scope. We used the generated Markdown structure as the basis of `BananaCare_Panabo_Mini_System_Proposal.md`. During later planning, we changed the location-specific scope into a reusable theoretical template, retained the farm-location field, and kept the three core modules: farmer and farm records, farm-condition monitoring, and assistance tracking.

Prompt Screenshot and Output :



Generated File:

BananaCare_Panabo_Mini_System_Proposal.md

Mini Information System Proposal: Banana Farmers in Panabo City

Recommended Theme

****Agriculture – Banana Farm Monitoring and Assistance System****

Agriculture is the better choice for this project because banana production is strongly associated with Panabo City and Davao del Norte, while documented local concerns include banana diseases, production losses, financial difficulties, and limited access to support and training. Panabo's City Agriculture Office has also conducted local efforts to address Fusarium wilt in banana farms.

Suggested System Title

****BananaCare: A Banana Farm Monitoring and Farmer Assistance Information System for Panabo City****

One-sentence title summary

A simple information system that helps banana farmers in selected areas of Panabo City record farm conditions, monitor banana problems, and keep track of assistance or recommended actions.

Target Location

The project should be limited to ****Panabo City, Davao del Norte****.

For a manageable study area, you can focus on selected banana-growing barangays such as:

- ****Barangay Kasilak**** – local records identify Cavendish banana as a major agricultural crop, with individual growers and planters.
- ****Barangay Little Panay**** – the Panabo City Agriculture Office conducted a Fusarium wilt mitigation trial involving Cavendish banana.
- ****Barangay Tibungol**** – a local study documented banana growers dealing with the financial costs and challenges associated with Sigatoka disease.

You do not need to cover the entire city; choosing one or two barangays would make the mini-system easier to develop and test.

Main Problem

****One-sentence problem summary:**** Banana farmers may have difficulty keeping organized records of farm conditions, disease incidents, production information, and assistance needs, making it harder to monitor problems and respond systematically.

Local evidence supports the relevance of this problem: studies involving Panabo banana growers report difficulties related to disease, weather, soil conditions, production, and financial costs, while regional stakeholders identify Fusarium wilt, rising production costs, and difficulty replacing affected farm areas as major banana-industry challenges.

System Planning Prompt

AI Tool Used: OpenAI Codex

Purpose of the Prompt:

The purpose of this prompt was to help us establish the overall structure, workflow, user roles, database choice, and main features of the BananaCare system before starting development.

What We Used/Changed from the AI Output:

We used the proposed Flask application structure and three-module workflow as a guide. We selected Python and Flask for the web application, PostgreSQL as the preferred database with SQLite as a local fallback, and separate Farmer and Admin login routes. We changed the original location-specific scope into a reusable theoretical template.

Prompt Screenshot and Output:

I am currently working on a mini system project about a **Banana Farm Monitoring and Assistance System** that is only required for a module assessment in our class. I want you to make the system based on the markdown file I've laid out in the folder, make it simple, you can ask me questions regarding some features and design elements while also giving me choices on what kind of programming language and framework we can use (though i would prefer it to be a web based app for easier access). The system requirements are already laid out in the markdown file. The system files should be laid out in the current folder and the expected output is shown in the pdf file.

I reviewed both files. The system needs three working modules, validation/error handling, at least three test cases, basic documentation, and localhost demonstration.

My recommended stack is:

- Python Flask
- SQLite database
- Bootstrap with custom banana-farm styling
- Server-rendered HTML, CSS, and JavaScript

This is the simplest option to install, demonstrate, understand, and explain during your presentation. Other reasonable choices are PHP + MySQL/XAMPP or Node.js + Express, but they add more setup.

Before I build it, please choose:

1. Stack: **Flask + SQLite (recommended)**, PHP + MySQL, or Node.js + SQLite?
2. Access: **One admin-style interface (simplest)**, or separate Farmer and Admin logins?
3. Coverage: **Kasilak and Little Panay (recommended)**, one barangay only, or all three listed barangays?

Unless you request something different, I'll use a clean green-and-yellow responsive design, include sample records, automated tests, setup instructions, and a documentation template aligned with the PDF. You will still need to capture screenshots of our actual prompts and outputs for the required AI-usage evidence.

Code Generation Prompt

AI Tool Used: OpenAI Codex

Purpose of the Prompt:

The purpose of this prompt was to generate the necessary code for the Farmer and Admin portals and the three major BananaCare modules while maintaining a simple project structure suitable for a classroom assessment.

What We Used/Changed from the AI Output:

We used the relevant generated Python, Flask, SQLAlchemy, HTML, Jinja, and CSS code. We reviewed the routes, model relationships, input handling, authentication rules, and page content. The implementation was adjusted to support a location-neutral farm template, separate user roles, local demonstration data, and the project requirements from the activity instructions.

Prompt Screenshot and Output:

I don't know what kind of dependencies and SDKs I have installed locally but if python works, we can use that, then for the database, I have postgresQL installed locally. for number 2, separate login routes might be better. for number 3, this is just theoretical so scope and location isn't actually required so let's just make it like a template

Implemented the complete BananaCare mini-system in the current folder.

Key features:

- Separate Farmer and Admin login routes
- Farmer registration and editable farm records
- Disease and farm-condition reporting
- Assistance request and recommendation tracking
- Admin monitoring and status updates
- Responsive green-and-yellow interface
- Password hashing, role protection, and input validation
- PostgreSQL support through `.env`
- SQLite fallback for immediate demonstration
- Demo accounts and sample records
- Seven automated test cases—all passing
- Assessment-aligned documentation

To run it immediately:

```
</> powershell  
python run.py
```

Then open `http://127.0.0.1:5000`.

Demo accounts:

- Admin: `admin` / `admin123`
- Farmer: `farmer1` / `farmer123`

The SQLite demonstration database has already been created and seeded.

For PostgreSQL, follow [README.md](#). You'll need to copy `.env.example` to `.env` and enter your PostgreSQL username and password.

Important files:

- [README.md](#) — installation and usage
- [DOCUMENTATION.md](#) — submission documentation template
- [run.py](#) — application launcher
- [tests/test_app.py](#) — seven test cases
- [models.py](#) — database structure
- [main.py](#) — main system features

Verification result: 7 tests passed, and the landing, Farmer login, Admin login, and registration pages all returned successful HTTP responses.



Edited 22 files
+760 -6

Undo ↶

Review

.env.example

+3 -0

.gitignore

+7 -0

bananacare/__init__.py

+45 -0

Show 19 more files ▾

Debugging Prompt

AI Tool Used: OpenAI Codex

Purpose of the Prompt:

The purpose of this prompt was to review the application during testing and verify that its required workflows behaved as expected.

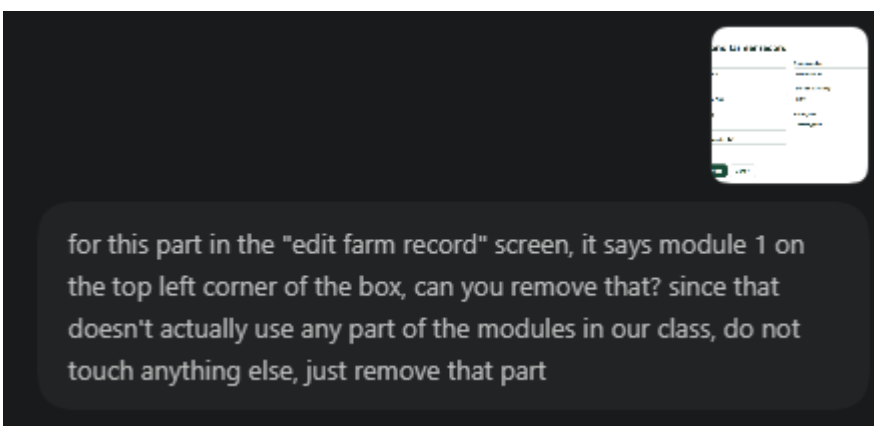
Testing Observation:

During AI-assisted development, the AI tool encountered an error while checking the application and corrected it immediately before presenting the completed output. Because the error was handled within the AI tool's working process, no separate user-facing screenshot of the error was captured. When the final version was tested, no bugs were found in the executed tests. The tested login, reporting, access-control, administrative, and assistance-request workflows produced their expected results. There were just some minor labelling errors that the user encountered and the AI tool fixed it afterwards

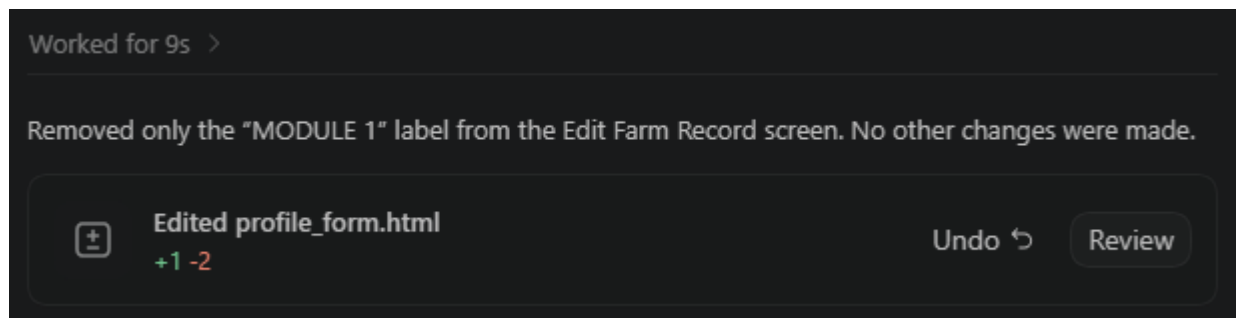
What We Used/Changed from the AI Output:

The corrected output provided by the AI tool was reviewed through the application's tests. The final executed tests passed, so no additional bug fix was required after the output was delivered. This result only applies to the scenarios that were tested and does not claim that the application is free of all possible defects.

Prompt Screenshot:



Output:



Refactoring Prompt

AI Tool Used: OpenAI Codex

Purpose of the Prompt:

The purpose of this prompt was to improve the readability, organization, maintainability, and consistency of the BananaCare source code without changing its intended behavior.

What We Used/Changed from the AI Output:

We centralized repeated status, role, problem-type, and assistance-type values in constants.py. We moved repeated form trimming and parsing into forms.py, improved names and formatting in the route modules, and updated the templates to use the centralized choices. The original version was retained in backups/bananacare-pre-refactor/, and both versions were tested to confirm that functionality was preserved.

Prompt Screenshot:

You originally built this entire system, so use your existing understanding of the codebase. Refactor the system for cleaner structure, better readability, maintainability, and consistency, while preserving all current functionality. Simplify complex code, remove duplication, improve naming and organization, and create/update clear documentation for the architecture, main modules, data flow, setup, and important functions. Do not change expected behavior, break existing features, or overengineer the solution. Keep changes practical and focused. Refactored code, updated documentation, and a short summary of the main changes made.

Output:

Refactoring is complete, and the original pre-refactor version was recovered successfully.

Main changes:

- Centralized roles, statuses, and selectable types in `constants.py`
- Moved repeated form parsing into reusable `forms.py` helpers
- Simplified and reformatted route functions
- Improved variable naming and model formatting
- Removed unused imports and duplicated literals
- Updated templates to use centralized choices
- Preserved every existing route, feature, database field, and workflow
- Added detailed architecture and data-flow documentation

The original version is preserved at:

[Pre-refactor backup](#)

Updated documentation:

BEFORE REFACTORING

```
from datetime import date
from functools import wraps

from flask import Blueprint, abort, flash, redirect, render_template, request, url_for
from flask_login import current_user, login_required

from . import db
from .models import Assistance, DiseaseReport, FarmerProfile, User

main_bp = Blueprint("main", __name__)

def role_required(role):
    def decorator(view):
        @wraps(view)
        @login_required
        def wrapped(*args, **kwargs):
            if current_user.role != role:
                abort(403)
            return view(*args, **kwargs)
        return wrapped
    return decorator

@main_bp.route("/")
def index():
    return render_template("index.html")

@main_bp.route("/dashboard")
@login_required
def dashboard():
    return redirect(url_for(f"main.{current_user.role}_dashboard"))

@main_bp.route("/farmer/dashboard")
@role_required("farmer")
def farmer_dashboard():
    profile = current_user.profile
    return render_template("farmer_dashboard.html", profile=profile,
        reports=DiseaseReport.query.filter_by(farmer_id=profile.id).order_by(DiseaseReport.id.desc()).all(),
        requests=Assistance.query.filter_by(farmer_id=profile.id).order_by(Assistance.id.desc()).all())
```

source: backup/bananacare/main.py

AFTER REFACTORING

```
from datetime import date
from functools import wraps

from flask import Blueprint, abort, flash, redirect, render_template, request, url_for
from flask_login import current_user, login_required

from . import db
from .constants import (
    ADMIN_ROLE,
    ASSISTANCE_STATUSES,
    ASSISTANCE_TYPES,
    FARMER_ROLE,
    PROBLEM_TYPES,
    REPORT_STATUSES,
)
from .forms import optional_date, optional_number, positive_number, text_field, text_fields
from .models import Assistance, DiseaseReport, FarmerProfile

main_bp = Blueprint("main", __name__)

def role_required(role):
    """Restrict a view to authenticated users with the required role."""
    def decorator(view):
        @wraps(view)
        @login_required
        def wrapped(*args, **kwargs):
            if current_user.role != role:
                abort(403)
            return view(*args, **kwargs)
        return wrapped
    return decorator

@main_bp.route("/")
def index():
    return render_template("index.html")

@main_bp.route("/dashboard")
@login_required
def dashboard():
    return redirect(url_for(f"main.{current_user.role}_dashboard"))
```

source: [bananacare/main.py](#)

OUR ANALYSIS AND MODIFICATIONS

Planning

Analysis – We reviewed the AI-generated system plan and compared it with the BananaCare proposal and activity requirements. We identified the necessary Farmer and Admin roles, the three required modules, the data each module needed, and the main workflow from farmer registration to Admin follow-up.

Modification – We selected Flask as the web framework and designed the system as a reusable template rather than limiting it to a specific barangay or location. We retained a free-text farm location field so the system can still be adapted to a real area later.

Code Generation

Analysis – We reviewed the generated application structure and checked how the models, routes, templates, login sessions, validation rules, and database connection worked together. We also compared every implemented feature with the requirements in the project proposal.

Modification – We adjusted the generated code to provide separate Farmer and Admin login routes, a responsive farm-themed interface, environment-based PostgreSQL configuration, an SQLite fallback, demonstration accounts, and automated tests. Only features within the required mini-system scope were included.

Debugging

Analysis – We examined the failed test import and verified that the application code was not the cause. The dependency installation had not completed before test discovery was attempted. We also reviewed a Python 3.13 warning and identified the outdated timestamp function.

Modification – We installed the requirements in a separate command, reran the tests, and confirmed that they passed. We replaced the deprecated UTC timestamp function with a timezone-aware implementation and ran the complete test suite again.

Refactoring

Analysis – We reviewed the existing source and identified repeated form-processing code, duplicated roles and status strings, dense route formatting, and hard-coded choices shared by routes and templates.

Modification – We added focused constants.py and forms.py modules, improved naming and layout in the route and model files, and made templates read their choices from shared constants. We avoided adding unnecessary design layers. The same seven tests passed in both the original and refactored versions.

TEST CASES

TESTING PROMPT AND OUTPUT:

I want to test the app for bugs myself, what kind of features do i need to test first?

Start with the main workflow and test both correct and incorrect inputs.

1. Farmer registration

Open `/farmer/register`.

Test:

- Complete registration with valid information
- Submit with empty required fields
- Enter `0`, a negative number, or text for farm size
- Use a password shorter than six characters
- Register an existing username
- Enter unusually long names or locations

Expected: valid accounts are created; invalid or duplicate data shows an error without crashing.

2. Login and access control

Test both portals:

- Correct Farmer credentials
- Correct Admin credentials
- Incorrect password
- Farmer credentials on the Admin login page
- Admin credentials on the Farmer login page
- Open `/admin/dashboard` while signed in as a Farmer
- Open dashboards while signed out
- Sign out, then use the browser Back button

Expected: users only access their authorized dashboard.

3. Farm profile

As a Farmer:

- Update every field with valid values
- Leave required fields blank
- Enter zero or negative farm size
- Enter invalid production values
- Set and remove the planting date
- Refresh after saving to confirm persistence

Expected: valid changes remain after refreshing; invalid changes are rejected.

4. Condition reports

Test:

- Submit each problem type
- Enter an affected area smaller than the farm size
- Enter an area equal to the farm size
- Enter zero, a negative value, or text
- Enter an area larger than the farm
- Leave the description empty
- Submit several reports

Expected: valid reports appear in history with `Reported` status. Invalid reports should not be saved.

5. Admin report monitoring

As Admin:

- View reports from different farmers
- Change `Reported` to `Under Monitoring`
- Add remarks
- Change the report to `Resolved`
- Sign back in as the Farmer and check the update
- Refresh to confirm status persistence

Expected: the Farmer sees the Admin's latest status and remarks.

6. Assistance tracking

As Farmer:

- Request every assistance type
- Submit with no assistance type
- Include and omit optional details
- Submit multiple requests

As Admin:

- Change Pending to In Progress
- Add or edit recommendations
- Mark it Completed
- Confirm that the provided date appears
- Change it back from Completed and check the date

7. Browser and interface checks

Test on:

- Desktop and narrow/mobile window widths
- Chrome, Edge, or Firefox
- Long descriptions and remarks
- Tables with many records
- Page refresh and browser navigation

Watch for overlapping buttons, cut-off text, unreadable tables, and horizontal overflow.

Priority test sequence

For the fastest useful check:

1. Register a new Farmer
2. Sign in as that Farmer
3. Update the farm profile
4. Submit a condition report
5. Submit an assistance request
6. Sign out and enter the Admin portal
7. Update both records
8. Return to the Farmer portal and verify the updates
9. Try invalid inputs and unauthorized pages

Record each issue with:

- Page and action
- Input used
- Expected result
- Actual result
- Screenshot or error text
- Whether it happens consistently

The existing automated tests can also be rerun with:

```
</> powershell  
python -m unittest discover -s tests -v
```

TESTING PHASE / TEST CASES

LOGIN AND REGISTRATION

Test Case	Test Scenario	Input	Expected Result	Actual Result	Status
1. Valid Input	Farmer logs in using a registered account with correct credentials.	Username: farmer1; Password: farmer123	The system accepts the credentials and redirects the Farmer to the Farmer dashboard.	The Farmer successfully logged in and was redirected to the Farmer dashboard.	PASS
2. Invalid Input	Admin attempts to log in using an incorrect password.	Username: admin; Password: incorrect password	The system rejects the login, displays an invalid-credentials message, and does not create a session.	The system rejected the login and displayed the error message.	PASS
3. Edge Case	A user attempts to register using an existing username.	Existing username with otherwise valid registration information	The system prevents duplicate registration and displays a duplicate-username message.	The duplicate account was not created and the error message was displayed.	PASS

FARMER

Test Case	Test Scenario	Input	Expected Result	Actual Result	Status
1. Valid Input	Farmer updates their farm profile.	Complete farmer details, positive farm size, valid planting date, and production value	The system saves the changes and shows them on the Farmer dashboard.	The profile was updated and the saved details remained after refresh.	PASS
2. Valid Input	Farmer submits a condition report.	Problem: Sigatoka; Affected area: 0.5 ha; Description: Leaf spots observed	The report is saved with Reported status and appears in the Farmer and Admin dashboards.	The report was saved and displayed with Reported status.	PASS
3. Invalid	Farmer submits an affected area	Farm size: 2 ha; Affected	The system rejects the report and displays a	The system displayed the affected-area error	PASS

Input	larger than the farm.	area: 3 ha	validation message. No report is created.	and did not save the report.	
4. Edge Case	Farmer requests assistance without optional details.	Assistance type: Farm Visit; Details: blank	The request is accepted because the details field is optional and begins with Pending status.	The request was saved successfully with Pending status.	PASS

ADMIN

Test Case	Test Scenario	Input	Expected Result	Actual Result	Status
1. Valid Input	Admin opens the monitoring dashboard.	Valid Admin username and password	The dashboard displays registered farmers, reports, and assistance requests.	The Admin dashboard opened and displayed the test farmer record.	PASS
2. Valid Input	Admin updates a condition report.	Status: Under Monitoring; Staff remarks: Follow-up farm inspection needed	The updated status and remarks are saved and become visible to the Farmer.	The report was updated and the changes appeared on the Farmer dashboard.	PASS
3. Valid Input	Admin completes an assistance request.	Status: Completed; Recommendation entered	The status and recommendation are saved and the current date is recorded as the date provided.	The assistance record was completed with a provided date.	PASS
4. Invalid Access	Farmer attempts to open the Admin dashboard.	Authenticated Farmer requests /admin/dashboard	The system denies access and returns HTTP 403.	The system returned HTTP 403 and did not display Admin information.	PASS

FINAL SYSTEM SCREENSHOTS

Landing Page

You have been signed out.

SIMPLE RECORDS. HEALTHIER FARMS.

Monitor banana farms and coordinate support in one place.

BananaCare is a lightweight information system for recording farm details, reporting field conditions, and tracking agricultural assistance.

Farmer portal

Admin portal



Three connected modules

Farm records · Condition monitoring · Assistance tracking

01

Farm Records

Maintain reusable farmer, location, variety, size, and production information.

02

Condition Reports

Document disease or pest observations and follow each case through resolution.

03

Assistance

Request farm visits or supplies and review recommendations from staff.

BananaCare · Farm Monitoring and Assistance System · Classroom Demo

Login / Registration

FARMER PORTAL

Welcome back

Sign in to continue to your farmer dashboard.

Username

Password

Sign in

New farmer? [Create an account](#)

[Use admin login instead](#)

BananaCare · Farm Monitoring and Assistance System · Classroom Demo

ADMIN PORTAL

Welcome back

Sign in to continue to your admin dashboard.

Username

Password

Sign in

[Use farmer login instead](#)

FARMER REGISTRATION

Create your farm record

Full name *

Contact number *

Farm location *

Farm size (hectares) *

Banana variety *

Username *

Password *

Register farmer

Farmer Side (Pages)



[Dashboard](#) [Sign out](#)

Welcome back!

FARMER DASHBOARD

Hello, Juan Dela Cruz

Edit farm profile

Sample Farm Area · Cavendish

Farm size

2.50 ha

Total reports

3

Open reports

2

Pending assistance

3

Farm condition reports

+ New report

Track reported diseases, pests, and other field concerns.

DATE	PROBLEM	AFFECTED	STATUS	STAFF REMARKS
2026-09-01	Other test	1.00 ha	Under Monitoring	test
2026-09-01	Fusarium Wilt asd	1.00 ha	Resolved	test
2026-09-01	Sigatoka Dark spots observed on several leaves.	0.40 ha	Reported	—

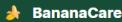
Assistance requests

+ Request assistance

View support requests and recommendations.

REQUESTED	TYPE	STATUS	DATE PROVIDED	RECOMMENDATION / NOTES
2026-09-01	Other	In Progress	—	test
2026-09-01	Seedlings	Pending	—	asd
2026-09-01	Farm Visit	Pending	—	Requesting inspection and disease-control advice.

BananaCare · Farm Monitoring and Assistance System · Classroom Demo



[Dashboard](#) [Sign out](#)

Farm and farmer record

Full name *

Juan Dela Cruz

Contact number *

09123456789

Farm location *

Sample Farm Area

Farm size (hectares) *

2.50

Banana variety *

Cavendish

Planting date

mm/dd/yyyy

Estimated production (kg)

Save changes

Cancel

BananaCare · Farm Monitoring and Assistance System · Classroom Demo

MODULE 2

Report a farm condition

Problem type *

Select a problem

Affected area (hectares) *

Description *

Describe what you observed...

Submit report

Cancel

MODULE 3

Request assistance

Assistance type *

Select assistance

Request details

Explain the support you need...

Submit request

Cancel

Complete every field. Affected area must be positive and cannot exceed farm size.

MODULE 2

Report a farm condition

Problem type *

Select a problem

Affected area (hectares) *

Description *

Describe what you observed...

Submit report

Cancel

BananaCare

Dashboard

Sign out

Welcome back!

AGRICULTURE STAFF PORTAL

Monitoring overview

Review farmer records, reported cases, and requested support.

Registered farmers

2

Total farm area

125.50 ha

Open reports

2

Pending support

3

Farmers and farms

Centralized registration records.

FARMER	CONTACT	LOCATION	SIZE	VARIETY
Juan Dela Cruz	09123456789	Sample Farm Area	2.50 ha	Cavendish
ruby jean indig	09123456789	IDK	123.00 ha	IDK

Condition monitoring

Update each case after review or follow-up.

Other

Juan Dela Cruz - 1.00 ha

Under Monitoring

test

Status

Under Monitoring

Staff remarks

test

Update report

Fusarium Wilt

Juan Dela Cruz - 1.00 ha

Resolved

asd

Status

Resolved

Staff remarks

test

Update report

Sigatoka

Juan Dela Cruz - 0.40 ha

Reported

Dark spots observed on several leaves.

Status

Reported

Staff remarks

Update report

Assistance tracking

Record support progress and recommendations.

Other

Juan Dela Cruz - 2026-09-01

In Progress

Status

In Progress

Recommendation / Notes

test

Update assistance

Seedlings

Juan Dela Cruz - 2026-09-01

Pending

Status

Pending

Recommendation / Notes

asd

Update assistance

Farm Visit

Juan Dela Cruz - 2026-09-01

Pending

Status

Pending

Recommendation / Notes

Requesting inspection and disease-control advice.

Update assistance

BananaCare · Farm Monitoring and Assistance System · Classroom Demo

Automated Testing

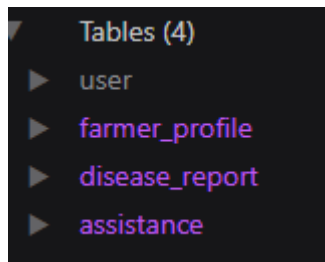
```
PS F:\4A\App Dev\Module 1-4 App> python -m unittest discover -s tests -v
test_admin_dashboard_renders (test_app.BananaCareTestCase.test_admin_dashboard_renders) ... ok
test_affected_area_cannot_exceed_farm_size (test_app.BananaCareTestCase.test_affected_area_cannot_exceed_farm_size) ... ok
test_assistance_request_is_saved (test_app.BananaCareTestCase.test_assistance_request_is_saved) ... ok
test_farmer_cannot_open_admin_dashboard (test_app.BananaCareTestCase.test_farmer_cannot_open_admin_dashboard) ... ok
test_invalid_login_shows_error (test_app.BananaCareTestCase.test_invalid_login_shows_error) ... ok
test_valid_condition_report_is_saved (test_app.BananaCareTestCase.test_valid_condition_report_is_saved) ... ok
test_valid_farmer_login (test_app.BananaCareTestCase.test_valid_farmer_login) ... ok
```

DATABASE

The default classroom version stores its records in the local SQLite database located at instance/bananacare.db. The database can be inspected using DB Browser for SQLite. PostgreSQL can also be used by configuring DATABASE_URL in the local .env file.

The database contains the following main tables:

- user – stores account credentials, roles, and creation dates.
- farmer_profile – stores farmer and farm information.
- disease_report – stores reported farm conditions, affected area, status, and Admin remarks.
- assistance – stores farmer assistance requests, progress, recommendations, and provided dates.



bananacare.db ▶ user						
Reset Filters		Records: 3				Search 3 rec
	id 🔑 📅 📌 Search column...	username 📄 📌 Search column...	password_ha... 📄 📌 Search column...	role 📄 📌 Search column...	created_at 📅 📌 Search column...	
1	1	admin	sCrypt:32768:8:1\$Y7...	admin	2026-09-01 10:41:4...	
2	2	farmer1	sCrypt:32768:8:1\$p0...	farmer	2026-09-01 10:41:4...	
3	3	indig	sCrypt:32768:8:1\$0...	farmer	2026-09-01 10:50:2...	

bananacare.db ▶ farmer_profile									
Reset Filters		Records: 2							
	id	user_id	full_name	contact_num...	location	farm_size	banana_variety	planting_date	estimated_pr...
	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...
1	1	2	Juan Dela Cruz	09123456789	Sample Farm Area	2.5	Cavendish	NULL	NULL
2	2	3	ruby jean indig	09123456789	IDK	123	IDK	NULL	NULL

bananacare.db ▶ disease_report								
Reset Filters		Records: 3						
	id	farmer_id	date_reported	problem_type	affected_area	description	status	remarks
	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...
1	1	1	2026-09-01	Sigatoka	0.4	Dark spots observe...	Reported	
2	2	1	2026-09-01	Fusarium Wilt	1	asd	Resolved	test
3	3	1	2026-09-01	Other	1	test	Under Monitoring	test

Reset Filters		Records: 3						
	id    	farmer_id  	assistance_ty...  	date_request...  	date_provided  	status  	recommenda...  	
	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...	Search column...	
	1	1	1	Farm Visit	2026-09-01	NULL	Pending	Requesting inspecti...
 	2	2	1	Seedlings	2026-09-01	NULL	Pending	asd
	3	3	1	Other	2026-09-01	NULL	In Progress	test